Sprint 08 Structured Notes: Memoization and Dependency Management

0. Where You Are Before Sprint 08

Before Sprint 08, you should already understand these React ideas:

- React builds user interfaces using components.
- JSX lets you write UI inside JavaScript.
- Props pass data from parent to child.
- `children` lets one component wrap other JSX.
- `map()` helps render lists.
- Stable keys help React track list items.
- Events let React respond to clicks, typing, and form submissions.
- Controlled inputs store form values in React state.
- `event.preventDefault()` stops a form from refreshing the page.
- `useState` lets React remember changing data.
- State updates should be immutable.
- Object state should be copied before changing fields.
- Array state should be updated with safe methods like spread, `filter()`, and `map()`.
- `useReducer` can organize related state actions.
- `useRef` can access DOM elements or store values that should not re-render the UI.
- `useEffect` runs side effects after React renders.
- Cleanup functions stop old timers, listeners, and subscriptions.
- Custom hooks let you reuse hook logic.
- Context API lets you share app-level values like user, theme, or language.
- Data fetching lets React communicate with APIs or backend services.
- Loading, data, and error states make async UI clear.
- `services/api.js` keeps API logic outside UI components.
- `useFetch` can reuse fetching logic.
- `useActionState` helps manage form/action result state.
- `useOptimistic` helps show instant UI feedback while async work completes.
- `startTransition` can mark some updates as lower priority.

Sprints 1–7 taught you how to build working React apps.

Sprint 08 starts a new question:

> Is my app doing unnecessary work?

Before this sprint, the main goal was correctness.

After this sprint, you begin thinking about efficiency and project package safety.

---

1. Sprint 08 Goal

The goal of Sprint 08 is to understand two main topics:

1. **Memoization**
2. **Dependency management**

Simple version:

> Sprint 08 teaches you how to avoid unnecessary calculations, avoid unnecessary child re-renders, and manage project packages safely.

By the end of Sprint 08, you should be able to:

- Explain memoization in simple words.
- Explain why React components re-render.
- Use `useMemo` for expensive calculated values.
- Use `React.memo` to skip unnecessary child component re-renders.
- Use `useCallback` to keep function references stable.
- Understand dependency arrays in `useMemo` and `useCallback`.
- Explain why memoization should not be used everywhere.
- Explain what a package is.
- Explain what `package.json` does.
- Explain what `package-lock.json` does.
- Explain what `node_modules` is.
- Explain the difference between `dependencies` and `devDependencies`.
- Install runtime packages and development packages correctly.
- Build a small Cart Summary app using `useMemo`, `useCallback`, and `React.memo`.

---

2. Big Mental Model

React apps often re-render.

A re-render means React runs a component function again to calculate what the UI should look like now.

A re-render can happen when:

- state changes,
- props change,
- a parent component re-renders,
- a user types in an input,
- a user clicks a button,
- fetched data is stored in state,
- context value changes.

Re-rendering is normal.

It is not automatically bad.

React needs re-rendering to keep the screen updated.

The problem is this:

> Sometimes a re-render causes extra work that was not needed.

Sprint 08 gives you tools for reducing that extra work.

---

3. What Is Memoization?

3.1 Simple Definition

Memoization means remembering a previous result and reusing it when the inputs have not changed.

Simple version:

> If React already knows the answer, and the data did not change, React can reuse the old answer instead of calculating again.

Example idea:

```
Same input
↓
Same output
↓
Reuse old result
```

3.2 Everyday Analogy

Imagine you calculate this once:

```
50 + 25 + 25 = 100
```

If the numbers do not change, there is no reason to calculate again.

You can remember:

```
Total = 100
```

Memoization works like that.

3.3 Why Memoization Exists in React

React components are functions.

When React re-renders a component, the function runs again.

That means normal JavaScript inside the component also runs again.

Example:

```
function CartSummary({ cartItems }) {
  const total = cartItems.reduce((sum, item) => {
    return sum + item.price * item.quantity;
  }, 0);

  return <p>Total: ${total}</p>;
}
```

This works.

But every time `CartSummary` re-renders, this calculation runs again:

```
const total = cartItems.reduce(...);
```

For a small cart, this is fine.

For a large cart or expensive calculation, it can waste time.

---

4. Unnecessary Work in React

4.1 Not All Re-Renders Are Bad

A beginner mistake is thinking:

> Re-rendering is always bad.

That is not true.

Re-rendering is how React updates the UI.

The real problem is unnecessary expensive work during re-renders.

4.2 Examples of Cheap Work

These are usually cheap:

```
const fullName = firstName + " " + lastName;
```

```
const isLoggedIn = user !== null;
```

```
const message = "Hello";
```

You usually do not need memoization for simple values like these.

4.3 Examples of Work That May Be Expensive

These can become expensive:

- filtering a very large list,
- sorting a very large list,
- calculating totals from many items,
- searching through thousands of records,
- rendering a complex child component many times,
- passing unstable function props to memoized child components.

Beginner rule:

> Memoize when there is real repeated work, not just because the tool exists.

---

5. The Three Main React Memoization Tools

Sprint 08 focuses on three tools:

Tool	Remembers	Main Use
`useMemo`	A calculated value	Avoid repeating expensive calculations

Tool	Remembers	Main Use
`useCallback`	A function reference	Keep function props stable
`React.memo`	A component render result	Skip child re-renders when props are unchanged

Simple memory rule:

```
useMemo     -> remembers a value
useCallback -> remembers a function
React.memo  -> remembers a component render when props stay the same
```

---

6. `useMemo`

6.1 What Is `useMemo`?

`useMemo` is a React Hook.

It remembers a calculated value.

Simple definition:

> `useMemo` lets React reuse a calculated value until its dependencies change.

6.2 Basic Syntax

```
const result = useMemo(() => {
  return expensiveCalculation(input);
}, [input]);
```

Read it like this:

> React, calculate this result. But only calculate it again when `input` changes.

6.3 Parts of `useMemo`

```
const result = useMemo(() => {
  return expensiveCalculation(input);
}, [input]);
```

Part	Meaning
`result`	The remembered calculated value
`useMemo`	React Hook for memoizing a value
`() => { ... }`	Function that calculates the value
`return expensiveCalculation(input)`	The calculation result
`[input]`	Dependency array

6.4 Without `useMemo`

```
import { useState } from "react";

function slowSquare(num) {
  console.log("Calculating square...");
  return num * num;
}

export default function App() {
  const [num, setNum] = useState(2);
  const [theme, setTheme] = useState("light");

  const square = slowSquare(num);

  return (
    <main>
      <h1>Without useMemo</h1>

      <p>Number: {num}</p>
      <p>Square: {square}</p>

      <button type="button" onClick={() => setNum(num + 1)}>
        Increase number
      </button>

      <button
        type="button"
        onClick={() => setTheme(theme === "light" ? "dark" : "light")}
      >
        Toggle theme
      </button>

      <p>Theme: {theme}</p>
    </main>
```

```
  );
}
```

What happens:

1. The component renders.
2. `slowSquare(num)` runs.
3. You click **Toggle theme**.
4. `theme` changes.
5. The component re-renders.
6. `slowSquare(num)` runs again, even though `num` did not change.

Problem:

> The square calculation repeats when only the theme changed.

6.5 With `useMemo`

```
import { useMemo, useState } from "react";

function slowSquare(num) {
  console.log("Calculating square...");
  return num * num;
}

export default function App() {
  const [num, setNum] = useState(2);
  const [theme, setTheme] = useState("light");

  const square = useMemo(() => {
    return slowSquare(num);
  }, [num]);

  return (
    <main>
      <h1>With useMemo</h1>

      <p>Number: {num}</p>
      <p>Square: {square}</p>

      <button type="button" onClick={() => setNum(num + 1)}>
        Increase number
      </button>

      <button
        type="button"
```

```
        onClick={() => setTheme(theme === "light" ? "dark" : "light")}
      >
        Toggle theme
      </button>

      <p>Theme: {theme}</p>
    </main>
  );
}
```

Now:

- Clicking **Increase number** recalculates the square.
- Clicking **Toggle theme** does not recalculate the square.

Why?

Because the dependency array is:

```
[num]
```

That means:

> Only recalculate `square` when `num` changes.

6.6 When to Use `useMemo`

Use `useMemo` when:

- a calculation is expensive,
- the calculation uses state or props,
- unrelated re-renders should not repeat the calculation,
- you are filtering, sorting, or reducing large data,
- a value is passed to a memoized child and must stay stable.

6.7 When Not to Use `useMemo`

Do not use `useMemo` for every value.

Bad example:

```
const message = useMemo(() => {
  return "Hello";
}, []);
```

Better:

```
const message = "Hello";
```

Bad example:

```
const fullName = useMemo(() => {
  return firstName + " " + lastName;
}, [firstName, lastName]);
```

Usually better:

```
const fullName = firstName + " " + lastName;
```

Why?

Because simple string joining is cheap.

Beginner rule:

> Do not make simple code complicated just to use `useMemo`.

---

7. Dependency Arrays

7.1 What Is a Dependency Array?

A dependency array tells React when to recalculate something.

You already saw dependency arrays in `useEffect`.

Example:

```
useEffect(() => {
  document.title = query;
}, [query]);
```

This means:

> Run this effect again when `query` changes.

In `useMemo`, the idea is similar.

Example:

```
const total = useMemo(() => {
  return cartItems.reduce((sum, item) => sum + item.price, 0);
}, [cartItems]);
```

This means:

> Recalculate `total` only when `cartItems` changes.

7.2 Important Dependency Rule

Every changing value used inside `useMemo` should usually be listed in the dependency array.

Bad:

```
const total = useMemo(() => {
  return cartItems.reduce((sum, item) => sum + item.price, 0) * taxRate;
}, [cartItems]);
```

Problem:

`taxRate` is used, but it is missing from the dependency array.

Better:

```
const total = useMemo(() => {
  return cartItems.reduce((sum, item) => sum + item.price, 0) * taxRate;
}, [cartItems, taxRate]);
```

7.3 What Happens If Dependencies Are Missing?

If a dependency is missing, React may reuse an old value.

That can create bugs where the UI shows stale data.

Simple definition:

> Stale data means old data that should have updated but did not.

7.4 Dependency Array Mental Model

```
Dependencies changed?
├── yes -> run calculation again
└── no  -> reuse previous result
```

7.5 Common Dependency Array Mistake

Bad:

```
const discountedTotal = useMemo(() => {
  return total - discount;
}, [total]);
```

Problem:

`discount` is used but missing.

Better:

```
const discountedTotal = useMemo(() => {
  return total - discount;
}, [total, discount]);
```

Beginner rule:

> If the memoized calculation reads it and it can change, include it.

---

8. `React.memo`

8.1 What Is `React.memo`?

`React.memo` is used to memoize a component.

Simple definition:

> `React.memo` lets React skip re-rendering a child component when its props have not changed.

8.2 Why Child Components Re-Render

When a parent component re-renders, child components may also re-render.

Example:

```
import { useState } from "react";

function Child() {
  console.log("Child rendered");
  return <p>I am the child</p>;
}

export default function App() {
  const [count, setCount] = useState(0);

  return (
    <main>
      <button type="button" onClick={() => setCount(count + 1)}>
        Count: {count}
      </button>

      <Child />
    </main>
  );
}
```

What happens:

1. User clicks the button.
2. `count` changes.
3. `App` re-renders.
4. `Child` may also re-render.

But `Child` did not receive any changing props.

So maybe rendering it again is unnecessary.

8.3 Using `React.memo`

```
import React, { useState } from "react";

const Child = React.memo(function Child() {
  console.log("Child rendered");
  return <p>I am the child</p>;
});

export default function App() {
  const [count, setCount] = useState(0);

  return (
    <main>
      <button type="button" onClick={() => setCount(count + 1)}>
        Count: {count}
      </button>

      <Child />
    </main>
  );
}
```

`React.memo` tells React:

> If this component receives the same props as last time, skip re-rendering it.

8.4 `React.memo` With Props

```
import React, { useState } from "react";

const UserCard = React.memo(function UserCard({ name }) {
  console.log("UserCard rendered");
  return <h2>{name}</h2>;
});

export default function App() {
  const [count, setCount] = useState(0);

  return (
```

```
    <main>
      <button type="button" onClick={() => setCount(count + 1)}>
        Count: {count}
      </button>

      <UserCard name="Sangeeth" />
    </main>
  );
}
```

Here, `UserCard` receives the same prop every time:

```
name="Sangeeth"
```

So `React.memo` can skip re-rendering `UserCard` when only `count` changes.

8.5 When to Use `React.memo`

Use `React.memo` when:

- the child component is expensive to render,
- the child receives props that often stay the same,
- the parent re-renders often,
- you have measured or noticed unnecessary child re-renders,
- the child is pure, meaning same props produce same UI.

8.6 When Not to Use `React.memo`

Do not wrap every component in `React.memo`.

Avoid it when:

- the component is small and cheap,
- props change almost every render,
- the code becomes harder to understand,
- you are using it without a real reason.

Beginner rule:

> `React.memo` is useful for expensive child components, not every component.

---

9. Function References and the Hidden Re-Render Problem

9.1 Functions Are Recreated During Render

This part is important.

Look at this code:

```
function App() {
  function handleAdd() {
    console.log("Add");
  }

  return <ChildButton onAdd={handleAdd} />;
}
```

Every time `App` re-renders, this function is created again:

```
function handleAdd() {
  console.log("Add");
}
```

Even if it has the same code, it is a new function reference.

9.2 What Is a Function Reference?

A function reference means the identity of a function in memory.

Two functions can look the same but still be different references.

Example:

```
const a = () => console.log("Hi");
const b = () => console.log("Hi");
```

`a` and `b` do the same thing.

But they are not the same function reference.

9.3 Why This Matters

If you pass a new function to a memoized child, the child sees the prop as changed.

That can break the benefit of `React.memo`.

Example problem:

```
const Child = React.memo(function Child({ onClick }) {
  console.log("Child rendered");
  return <button onClick={onClick}>Click</button>;
});

export default function App() {
  const [count, setCount] = useState(0);

  return (
    <main>
      <button type="button" onClick={() => setCount(count + 1)}>
        Parent count: {count}
      </button>

      <Child onClick={() => console.log("Child clicked")} />
    </main>
  );
}
```

Problem:

```
onClick={() => console.log("Child clicked")}
```

This creates a new function every render.

So even though `Child` uses `React.memo`, its `onClick` prop changes every time.

---

10. `useCallback`

10.1 What Is `useCallback`?

`useCallback` is a React Hook.

It remembers a function reference.

Simple definition:

> `useCallback` keeps the same function reference until its dependencies change.

10.2 Basic Syntax

```
const handleClick = useCallback(() => {
  console.log("Clicked");
}, []);
```

Read it like this:

> React, keep this function stable unless something in the dependency array changes.

10.3 Parts of `useCallback`

```
const handleClick = useCallback(() => {
  console.log("Clicked");
}, []);
```

Part	Meaning
`handleClick`	The remembered function
`useCallback`	React Hook for memoizing function references
`() => { ... }`	The function logic
`[]`	Dependency array

10.4 Example With `React.memo` and `useCallback`

```
import React, { useCallback, useState } from "react";

const AddButton = React.memo(function AddButton({ onAdd }) {
  console.log("AddButton rendered");

  return (
    <button type="button" onClick={onAdd}>
      Add
    </button>
  );
});
```

```
export default function App() {
  const [count, setCount] = useState(0);
  const [theme, setTheme] = useState("light");

  const handleAdd = useCallback(() => {
    setCount((prevCount) => prevCount + 1);
  }, []);

  return (
    <main>
      <h1>useCallback Example</h1>

      <p>Count: {count}</p>
      <p>Theme: {theme}</p>

      <AddButton onAdd={handleAdd} />

      <button
        type="button"
        onClick={() => setTheme(theme === "light" ? "dark" : "light")}
      >
        Toggle theme
      </button>
    </main>
  );
}
```

What this does:

- `AddButton` is memoized with `React.memo`.
- `handleAdd` is memoized with `useCallback`.
- When `theme` changes, `handleAdd` keeps the same reference.
- Because the `onAdd` prop stays the same, `AddButton` can skip re-rendering.

10.5 Why the Dependency Array Is Empty

In this code:

```
const handleAdd = useCallback(() => {
  setCount((prevCount) => prevCount + 1);
}, []);
```

The dependency array is empty because the function does not directly read `count`.

It uses an updater function:

```
setCount((prevCount) => prevCount + 1);
```

This is safe because React gives the latest previous count.

10.6 When Dependencies Are Needed in `useCallback`

If the callback reads a changing value, include it.

Example:

```
const handleLogUser = useCallback(() => {
  console.log(user.name);
}, [user]);
```

Why?

Because the function uses `user`.

If `user` changes, the function should update.

10.7 When to Use `useCallback`

Use `useCallback` when:

- you pass a function to a memoized child component,
- a function is used inside a dependency array,
- recreating the function causes unnecessary work,
- a child depends on stable function props.

10.8 When Not to Use `useCallback`

Do not wrap every event handler in `useCallback`.

Bad habit:

```
const handleClick = useCallback(() => {
  console.log("Clicked");
}, []);
```

This is unnecessary if:

- the function is not passed to a memoized child,

- the component is simple,
- there is no performance problem.

Beginner rule:

> Use `useCallback` mostly when function identity matters.

---

11. `useMemo` vs `useCallback`

`useMemo` and `useCallback` are related, but they are not the same.

Tool	Returns	Use It For
`useMemo`	A value	Expensive calculated results
`useCallback`	A function	Stable function references

11.1 `useMemo` Example

```
const total = useMemo(() => {
  return price * quantity;
}, [price, quantity]);
```

This remembers a value:

```
total
```

11.2 `useCallback` Example

```
const handleClick = useCallback(() => {
  console.log("Clicked");
}, []);
```

This remembers a function:

```
handleClick
```

11.3 Simple Memory Rule

```
useMemo     = remember the result
useCallback = remember the function
```

---

12. How `React.memo`, `useMemo`, and `useCallback` Work Together

These tools often work together.

Example situation:

- Parent re-renders often.
- Child is wrapped in `React.memo`.
- Parent passes a calculated value to child.
- Parent passes a function to child.

To optimize this:

- Use `useMemo` for the calculated value.
- Use `useCallback` for the function.
- Use `React.memo` for the child component.

Mental model:

```
Parent re-renders
↓
useMemo keeps calculated value stable when dependencies do not change
↓
useCallback keeps function stable when dependencies do not change
↓
React.memo lets child skip re-render when props stay the same
```

But remember:

> These tools are useful only when there is actual unnecessary work.

---

13. Full Sprint 08 Example: Cart Summary

This example combines:

- state,
- list rendering,
- immutable array updates,
- `useMemo`,
- `useCallback`,
- `React.memo`,
- controlled input,
- stable keys,
- console logs for observation.

13.1 Full Code

```
import React, { useCallback, useMemo, useState } from "react";

const CheckoutButton = React.memo(function CheckoutButton({ onCheckout }) {
  console.log("CheckoutButton rendered");

  return (
    <button type="button" onClick={onCheckout}>
      Checkout
    </button>
  );
});

export default function App() {
  const [cartItems, setCartItems] = useState([
    { id: "p1", name: "Keyboard", price: 50, quantity: 1 },
    { id: "p2", name: "Mouse", price: 25, quantity: 2 },
  ]);

  const [coupon, setCoupon] = useState("");

  const total = useMemo(() => {
    console.log("Calculating total...");

    return cartItems.reduce((sum, item) => {
      return sum + item.price * item.quantity;
    }, 0);
  }, [cartItems]);

  const handleCheckout = useCallback(() => {
    console.log("Checking out...");
```

```
}, []);

function addItem() {
  const newItem = {
    id: crypto.randomUUID(),
    name: "USB Cable",
    price: 10,
    quantity: 1,
  };

  setCartItems((prevItems) => [...prevItems, newItem]);
}

return (
  <main>
    <h1>Cart Summary</h1>

    <section>
      <h2>Items</h2>

      <ul>
        {cartItems.map((item) => (
          <li key={item.id}>
            {item.name} - ${item.price} × {item.quantity}
          </li>
        ))}
      </ul>

      <button type="button" onClick={addItem}>
        Add USB Cable
      </button>
    </section>

    <section>
      <h2>Total</h2>
      <p>${total}</p>
    </section>

    <section>
      <h2>Coupon</h2>

      <label htmlFor="coupon">Coupon code</label>
      <input
        id="coupon"
        value={coupon}
        onChange={(event) => setCoupon(event.target.value)}
      />
```

```
        <p>Coupon typed: {coupon}</p>
      </section>

      <CheckoutButton onCheckout={handleCheckout} />
    </main>
  );
}
```

13.2 What This Example Teaches

`cartItems` State

```
const [cartItems, setCartItems] = useState([...]);
```

This stores the cart items.

When cart items change, React re-renders the app.

`coupon` State

```
const [coupon, setCoupon] = useState("");
```

This stores the controlled input value.

Typing in the coupon field changes `coupon`.

`useMemo` for Total

```
const total = useMemo(() => {
  console.log("Calculating total...");

  return cartItems.reduce((sum, item) => {
    return sum + item.price * item.quantity;
  }, 0);
}, [cartItems]);
```

This calculates the total only when `cartItems` changes.

Typing in the coupon input should not recalculate the total.

`useCallback` for Checkout

```
const handleCheckout = useCallback(() => {
  console.log("Checking out...");
}, []);
```

This keeps the checkout function stable.

`React.memo` for `CheckoutButton`

```
const CheckoutButton = React.memo(function CheckoutButton({ onCheckout }) {
  console.log("CheckoutButton rendered");

  return (
    <button type="button" onClick={onCheckout}>
      Checkout
    </button>
  );
});
```

This lets `CheckoutButton` skip re-rendering when its props do not change.

`addItem` Uses Immutable Update

```
setCartItems((prevItems) => [...prevItems, newItem]);
```

This creates a new array instead of mutating the old array.

This follows the Sprint 05 state rule.

13.3 What to Observe in the Console

Open the browser console.

When the app first loads, you may see:

```
Calculating total...
CheckoutButton rendered
```

When you type in the coupon field:

```
Calculating total...
```

should not run again.

Why?

Because `cartItems` did not change.

When you click **Add USB Cable**, you should see:

```
Calculating total...
```

Why?

Because `cartItems` changed.

When you type in the coupon field, `CheckoutButton` should not keep rendering unnecessarily if `onCheckout` stays stable.

---

14. Common Memoization Mistakes

Mistake 1: Using `useMemo` Everywhere

Bad:

```
const message = useMemo(() => {
  return "Hello";
}, []);
```

Better:

```
const message = "Hello";
```

Why?

The value is cheap and simple.

Mistake 2: Missing Dependencies

Bad:

```
const total = useMemo(() => {
  return price * quantity * taxRate;
}, [price, quantity]);
```

Problem:

`taxRate` is missing.

Better:

```
const total = useMemo(() => {
  return price * quantity * taxRate;
}, [price, quantity, taxRate]);
```

Mistake 3: Using `React.memo` but Passing a New Function Every Render

Bad:

```
const Child = React.memo(function Child({ onClick }) {
  return <button onClick={onClick}>Click</button>;
});

function App() {
  return <Child onClick={() => console.log("Clicked")} />;
}
```

Problem:

This creates a new function every render:

```
() => console.log("Clicked")
```

Better:

```
const handleClick = useCallback(() => {
  console.log("Clicked");
}, []);

return <Child onClick={handleClick} />;
```

Mistake 4: Thinking `React.memo` Stops All Renders

`React.memo` does not stop all renders.

It only helps when props are the same.

If props change, the component should re-render.

Mistake 5: Memoizing Before Understanding the Problem

Bad process:

```
App feels normal
↓
Add useMemo/useCallback everywhere
↓
Code becomes harder to read
```

Better process:

```
Notice or measure repeated expensive work
↓
Identify the calculation/component/function causing it
↓
Apply the correct memoization tool
```

Mistake 6: Ignoring Readability

Memoization can make code harder to read.

Do not trade simple code for complicated code unless there is a real benefit.

Beginner rule:

> Correct code first. Clear code second. Optimize when needed.

---

15. Dependency Management

Sprint 08 also teaches how React projects manage external packages.

15.1 What Is a Package?

A package is reusable code that someone else created and published.

You can install packages into your project.

Examples:

```
react
react-dom
vite
eslint
react-router-dom
axios
vitest
```

Simple definition:

> A package is external code your project can use.

15.2 Why Packages Are Useful

Packages help you avoid writing everything from scratch.

Examples:

- React gives component-based UI tools.
- Vite gives development and build tooling.
- React Router gives routing.
- Axios helps with HTTP requests.
- Vitest helps with testing.
- ESLint helps catch code issues.

15.3 Why Packages Need Care

Packages are useful, but they add responsibility.

Problems can happen when:

- too many packages are installed,

- packages are outdated,
- packages have breaking changes,
- packages have security issues,
- packages are abandoned,
- package versions conflict,
- package size makes the app heavier.

Beginner rule:

> Install packages only when they solve a real problem.

---

16. `package.json`

16.1 What Is `package.json`?

`package.json` is the main information file for a JavaScript project.

Simple definition:

> `package.json` tells npm what the project is, what scripts it has, and what packages it uses.

It usually contains:

- project name,
- project version,
- scripts,
- dependencies,
- devDependencies.

16.2 Example `package.json`

```
{
  "scripts": {
    "dev": "vite",
    "build": "vite build"
  },
  "dependencies": {
    "react": "^19.0.0",
    "react-dom": "^19.0.0"
  },
  "devDependencies": {
    "@vitejs/plugin-react": "latest",
    "vite": "latest"
```

```
  }
}
```

16.3 Scripts

Scripts are commands you can run with npm.

Example:

```
"scripts": {
  "dev": "vite",
  "build": "vite build"
}
```

Run the dev script:

```
npm run dev
```

Run the build script:

```
npm run build
```

Simple definition:

> Scripts are shortcut commands stored in `package.json`.

---

17. `dependencies`

17.1 What Are `dependencies`?

`dependencies` are packages your app needs while it runs.

Example:

```
"dependencies": {
  "react": "^19.0.0",
  "react-dom": "^19.0.0",
  "react-router-dom": "^7.0.0",
```

```
  "axios": "^1.0.0"
}
```

These are runtime packages.

Simple definition:

> `dependencies` are packages needed by the actual application.

17.2 Examples of Runtime Dependencies

Common runtime dependencies:

```
react
react-dom
react-router-dom
axios
zustand
```

Why?

Because the app may need them while running in the browser.

17.3 Beginner Rule

> If the app needs it while running, it probably belongs in `dependencies`.

---

18. `devDependencies`

18.1 What Are `devDependencies`?

`devDependencies` are packages needed only for development, testing, linting, or building.

Example:

```
"devDependencies": {
  "vite": "latest",
  "eslint": "latest",
  "vitest": "latest"
}
```

Simple definition:

> `devDependencies` are tools developers need while building the app.

18.2 Examples of Development Dependencies

Common development dependencies:

```
vite
eslint
vitest
prettier
@vitejs/plugin-react
```

These are not normal app features.

They help developers build, test, format, or check the app.

18.3 Beginner Rule

> If it is only a development tool, testing tool, linter, formatter, or build tool, it usually belongs in `devDependencies`.

---

19. `dependencies` vs `devDependencies`

Category	Used For	Examples
`dependencies`	Running the app	`react`, `react-dom`, `axios`, `react-router-dom`
`devDependencies`	Developing, testing, building	`vite`, `eslint`, `vitest`, `prettier`

Simple memory rule:

```
dependencies    -> app needs them
devDependencies -> developer tools need them
```

Common beginner mistake:

> Putting everything in `dependencies`.

Better:

> Put runtime packages in `dependencies` and development-only tools in `devDependencies`.

---

20. `node_modules`

20.1 What Is `node_modules`?

`node_modules` is the folder where npm installs package code.

When you run:

```
npm install
```

npm downloads packages and puts them in:

```
node_modules/
```

Simple definition:

> `node_modules` stores the installed package files.

20.2 Should You Edit `node_modules`?

No.

You usually do not edit `node_modules` manually.

Why?

Because it is generated by npm.

If you delete it, you can usually recreate it with:

```
npm install
```

20.3 Should You Commit `node_modules`?

Usually no.

Most projects do not commit `node_modules` to Git.

Why?

Because it can be very large and can be recreated from `package.json` and `package-lock.json`.

---

21. `package-lock.json`

21.1 What Is `package-lock.json`?

`package-lock.json` records the exact installed versions of packages.

Simple definition:

> `package-lock.json` locks the exact package versions used by the project.

21.2 Why `package-lock.json` Matters

It helps make installs predictable.

Without a lock file, different computers may install slightly different versions.

With a lock file:

- your computer installs the same versions,
- another developer installs the same versions,
- deployment installs the same versions,
- builds are more predictable.

21.3 Should You Edit `package-lock.json` Manually?

Usually no.

Let npm update it.

Commands like these can update it:

```
npm install
npm update
```

Beginner rule:

> Commit `package-lock.json`, but do not manually edit it unless you know exactly what you are doing.

---

22. Installing Packages

22.1 Install a Runtime Package

Example:

```
npm install axios
```

This adds `axios` to `dependencies`.

Use this when the app needs the package while running.

22.2 Install a Development Package

Example:

```
npm install -D vitest
```

The `-D` means development dependency.

This adds `vitest` to `devDependencies`.

Same command written longer:

```
npm install --save-dev vitest
```

22.3 Install All Packages in a Project

When you download or clone a project, run:

```
npm install
```

This reads:

```
package.json
package-lock.json
```

Then it recreates:

```
node_modules/
```

22.4 Update Packages

```
npm update
```

This updates packages according to allowed version ranges.

Be careful.

Package updates can introduce breaking changes.

Beginner rule:

> Do not randomly update everything right before an important deadline.

---

23. Choosing Packages Carefully

Before installing a package, ask:

1. Do I really need this package?
2. Can I solve the problem with built-in browser or React features?
3. Is the package commonly used?
4. Is it maintained?
5. Is it too large for the small problem I have?
6. Does it fit my project?
7. Does it add unnecessary complexity?

Good package decision:

```
Problem is real
↓
Package solves it clearly
↓
Package is maintained
↓
Install it
```

Bad package decision:

```
Package looks interesting
↓
Install it without need
↓
Project becomes heavier and harder to maintain
```

Beginner rule:

> Every package is a dependency you must trust and maintain.

---

24. Common Dependency Management Mistakes

Mistake 1: Installing Packages Without Need

Bad:

```
Need one small helper function
↓
Install a large package
```

Better:

```
Write the small helper yourself
```

Mistake 2: Confusing `dependencies` and `devDependencies`

Bad mental model:

```
Everything goes in dependencies
```

Better mental model:

```
App runtime package -> dependencies
Development tool -> devDependencies
```

Mistake 3: Editing `node_modules`

Bad:

```
Open node_modules and change package code manually
```

Problem:

Your changes can disappear when packages are reinstalled.

Better:

```
Do not manually edit node_modules
```

Mistake 4: Deleting `package-lock.json` Randomly

Bad:

```
Delete package-lock.json to fix confusion
```

This can make installs less predictable.

Better:

```
Understand the package issue first
```

Mistake 5: Updating Everything Without Testing

Bad:

```
npm update
```

Then not testing the app.

Better:

```
Update carefully
Run the app
Run tests if available
Check for broken behavior
```

---

25. Debug Task 1: Bad `useMemo`

Broken Code

```
const total = useMemo(() => {
  return price * quantity * taxRate;
}, [price, quantity]);
```

Problem

`taxRate` is used inside the calculation but missing from the dependency array.

Fixed Code

```
const total = useMemo(() => {
  return price * quantity * taxRate;
}, [price, quantity, taxRate]);
```

Explanation

If `taxRate` changes, React should recalculate `total`.

---

26. Debug Task 2: Unnecessary `useMemo`

Broken Code

```
const message = useMemo(() => {
  return "Welcome";
}, []);
```

Problem

This value is simple and cheap.

`useMemo` adds unnecessary complexity.

Fixed Code

```
const message = "Welcome";
```

---

27. Debug Task 3: `React.memo` With Unstable Function Prop

Broken Code

```
const SaveButton = React.memo(function SaveButton({ onSave }) {
  console.log("SaveButton rendered");
  return <button onClick={onSave}>Save</button>;
});

export default function App() {
  const [theme, setTheme] = useState("light");

  return (
    <main>
      <SaveButton onSave={() => console.log("Saved")} />

      <button onClick={() => setTheme(theme === "light" ? "dark" : "light")}>
        Toggle theme
      </button>
```

```
    </main>
  );
}
```

Problem

This creates a new function every render:

```
() => console.log("Saved")
```

So `SaveButton` receives a changed prop every time.

Fixed Code

```
import React, { useCallback, useState } from "react";

const SaveButton = React.memo(function SaveButton({ onSave }) {
  console.log("SaveButton rendered");
  return <button type="button" onClick={onSave}>Save</button>;
});

export default function App() {
  const [theme, setTheme] = useState("light");

  const handleSave = useCallback(() => {
    console.log("Saved");
  }, []);

  return (
    <main>
      <SaveButton onSave={handleSave} />

      <button
        type="button"
        onClick={() => setTheme(theme === "light" ? "dark" : "light")}
      >
        Toggle theme
      </button>
    </main>
  );
}
```

---

28. Sprint 08 Practice Tasks

28.1 Warm-Up Drills

Practice these from memory:

1. Write a one-sentence definition of memoization.
2. Write the basic `useMemo` syntax.
3. Write the basic `useCallback` syntax.
4. Wrap a child component in `React.memo`.
5. Explain the difference between `dependencies` and `devDependencies`.
6. Write the command to install a runtime package.
7. Write the command to install a development package.

28.2 Guided Coding Task

Build a **Cart Summary** app.

Requirements:

- Store cart items in state.
- Render cart items with `map()`.
- Use stable keys.
- Calculate cart total with `useMemo`.
- Add a coupon input controlled by state.
- Prove typing in the coupon input does not recalculate the total.
- Add a memoized `CheckoutButton`.
- Use `useCallback` for the checkout handler.

28.3 Build Something Small

Build a **Filtered Products** app.

Requirements:

- Store a list of products.
- Store a search input value.
- Use `useMemo` to calculate filtered products.
- Show only products matching the search.
- Add a separate theme toggle.
- Confirm theme changes do not recalculate filtered products unless the search or product list changes.

28.4 Challenge Task

Build a **Performance Practice Dashboard**.

Requirements:

- Create a list of at least 100 fake items.
- Add a search input.
- Add a category filter.
- Use `useMemo` for filtered results.
- Create a memoized child list component with `React.memo`.
- Pass a stable callback to the child with `useCallback`.
- Add console logs to observe renders and recalculations.

---

29. Revision Checkpoint Questions

Answer these without looking at the notes:

1. What does memoization mean?
2. Why do React components re-render?
3. Is every re-render bad?
4. What does `useMemo` remember?
5. What does `useCallback` remember?
6. What does `React.memo` do?
7. When should you use `useMemo`?
8. When should you avoid `useMemo`?
9. Why can missing dependencies cause stale values?
10. Why can a function prop cause a memoized child to re-render?
11. How does `useCallback` help with function props?
12. What is a package?
13. What does `package.json` store?
14. What is `node_modules`?
15. What does `package-lock.json` do?
16. What is the difference between `dependencies` and `devDependencies`?
17. What does `npm install axios` do?
18. What does `npm install -D vitest` do?
19. Why should you be careful when updating packages?
20. What is the shortest mental model for Sprint 08?

---

30. Sprint 08 Definition of Done

You are ready to move past Sprint 08 when you can do these:

- Explain memoization in simple words.
- Explain that re-rendering is normal in React.
- Identify unnecessary repeated calculations.
- Use `useMemo` for an expensive calculated value.
- Use a correct dependency array with `useMemo`.
- Explain what stale data means.
- Use `React.memo` to memoize a child component.
- Explain why `React.memo` is not needed for every component.
- Use `useCallback` to keep a function prop stable.
- Explain the difference between `useMemo` and `useCallback`.
- Build the Cart Summary app.
- Use console logs to prove when a calculation runs.
- Use console logs to prove when a child component renders.
- Explain what a package is.
- Explain `package.json`.
- Explain `package-lock.json`.
- Explain `node_modules`.
- Explain `dependencies`.
- Explain `devDependencies`.
- Install runtime and development packages correctly.

---

31. Sprint 08 Short Summary

Sprint 08 is about avoiding unnecessary work and managing project packages safely.

The React optimization tools are:

```
useMemo     -> remembers calculated values
useCallback -> remembers function references
React.memo  -> skips child re-renders when props are unchanged
```

The dependency management tools are:

```
package.json      -> project info, scripts, dependencies
dependencies      -> packages needed by the running app
devDependencies   -> packages needed only for development
```

```
node_modules      -> installed package files
package-lock.json -> exact installed package versions
```

Shortest correct mental model:

> `useMemo` remembers values. `useCallback` remembers functions. `React.memo` helps child components skip re-renders when props stay the same. Dependency management keeps external project code organized, predictable, and safer to maintain.

---

32. Sprint Retrospective Prompts

Use these prompts after finishing the practice tasks:

1. What felt easy in Sprint 08?
2. What felt confusing?
3. Which tool is clearest: `useMemo`, `useCallback`, or `React.memo`?
4. Which tool needs more practice?
5. Did you use memoization only where it helped?
6. Did any memoization make the code harder to read?
7. Can you explain the Cart Summary example without looking?
8. Can you explain the difference between runtime packages and development packages?
9. What would you refactor before moving to Sprint 09?
10. What small app should you build once more before continuing?